Implementation Plan: Finance & Habit Pattern Finder

Topic 9 – Google Opal

1. Project Overview

Project Name: Finance & Habit Pattern Finder

Platform: Google Opal

Implementation Approach: Input Collection → Memory Retrieval → Transaction Analysis → Memory Update → Financial Report

The project uses Google Opal workflow nodes to collect financial transaction details, analyze historical spending records, update persistent memory, and generate a browser-based financial insights report.

2. Workflow Components

2.1 Transaction Date Input

Captures the date of the user's transaction.

2.2 Amount Input

Captures the numerical transaction amount.

2.3 Category Input

Captures the spending category, such as Food, Transport, Shopping, or Utilities.

2.4 Description Input

Captures a short description of the transaction.

2.5 Process Transaction Analysis

This Generate node performs the main analysis.

Responsibilities:

- Retrieve historical transaction records using the Use Memory tool.
- Handle an empty initial transaction history.
- Compare the latest transaction with relevant historical transactions.
- Identify spending increases, repeated categories, and patterns.
- Calculate transaction totals and differences.
- Generate personalized recommendations.
- Append the current transaction to historical memory without overwriting earlier records.

2.6 Generate Financial Report

This Generate node converts the analysis results into a financial dashboard.

Responsibilities:

- Display the latest transaction.
- Present spending patterns.
- Generate a personalized action plan.
- Display weekly financial insights based on available records.
- Display the complete available transaction archive.
- Report memory status accurately.

3. Workflow Connection Design

The workflow follows this structure:

```
None
Transaction Date ─┐
Amount ───────────┤
Category ─────────┼──> Process Transaction Analysis
Description ──────┘                |
                                   |
                            Use Memory Tool
                                   |
                                   v
                          Generate Financial Report
                                   |
                                   v
                             Browser Output
```

4. Persistent Memory Configuration

The Use Memory tool is configured within the Process Transaction Analysis node.

Memory operations:

1. Retrieve the existing transaction history.
2. Initialize an empty list when no history exists.
3. Preserve previous transaction records.
4. Append the current transaction exactly once.
5. Save the complete updated history.
6. Verify the update where supported.

Example transaction structure:

```
None
{
  "transactions": [
    {
      "date": "2026-11-06",
      "amount": 120,
      "category": "Food",
      "description": "Lunch at restaurant"
    }
  ]
}
```

5. Financial Analysis Logic

The analysis process compares transactions where suitable historical records exist.

Example:

Previous similar lunch: $120

Current lunch: $180

Difference:

```
None
$180 - $120 = $60
```

Percentage increase:

```
None
($60 / $120) * 100 = 50%
```

The system also calculates total spending from available records.

Example:

```
None
$120 + $450 + $180 = $750
```

6. Dashboard Design

The final browser output includes:

- Personal Finance Intelligence header.
- Latest Transaction section.
- Spending Patterns section.
- Personalized Financial Action Plan.
- Weekly Financial Insights.
- Transaction Archive.
- Memory Status.
- Project footer.

The dashboard uses a responsive fintech-style design with readable sections and transaction tables.

7. Validation and Quality Controls

The implementation includes the following controls:

- Use only supplied inputs and retrieved historical data.
- Preserve transaction dates, amounts, categories, and descriptions.
- Do not invent missing transactions or spending benchmarks.
- Avoid unsupported habit conclusions from insufficient records.

- Display all available transactions in the archive.
- Validate arithmetic in spending comparisons and totals.
- Report memory success only when confirmed by the workflow.

8. Implementation Status

The main workflow and three transaction test scenarios have been executed.

The final archive display was corrected to show all three sample transactions.

The weekly insight section is included in the reporting instructions and should be verified in the final preview before submission.

9. Conclusion

The implementation demonstrates the use of Google Opal to combine user input nodes, AI-based transaction analysis, persistent memory, and browser-based financial reporting in one workflow.